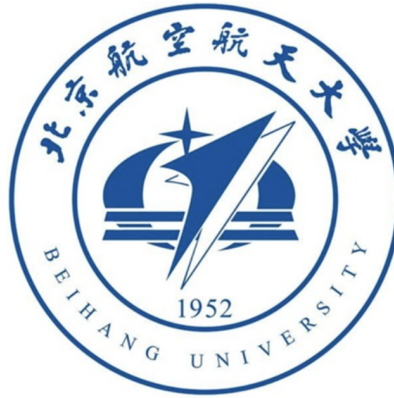


Beihang University

Course: OPTIMAL ESTIMATION



**Midterm assignment:
Accelerometer Static Calibration**

Professor: Zhang Hai

Student: Riccardo Caccia LJ2615105

Contents

1	Introduction	2
2	Measurement Models Overview	2
3	Linear Calibration Method	3
3.1	True Acceleration Computation	3
3.2	Accelerometer Measurement Simulation	4
3.3	Ordinary Least Square Algorithm	5
3.4	Results	6
3.5	Sensitivity Analysis	8
4	Non-Linear Calibration Method	9
4.1	Cost Function Formulation	9
4.2	Optimization Method	10
4.3	Results	12
4.4	Sensitivity Analysis	13
5	Conclusions	16

1 Introduction

In the modern aerospace sector, sensors like accelerometers play a crucial role in providing information about real-time motion of a vehicle. However, in real world, this type of sensors cannot produce perfect measurements because they are always affected by stochastic and deterministic errors and this is the reason why filtering and calibration are consistently required. This work will focus on the estimation of deterministic errors, such as biases and scale factors, which affect the measurement of a real-world three-axes accelerometer subjected only to gravitational acceleration.

2 Measurement Models Overview

The simulation of a real world sensor must rely on a measurement model. This model accounts for all the errors that need to be modeled to correctly represent the sensor's output. Following the requirements for this work it is possible to present the fundamental accelerometer measurement model by applying scale factors and biases to the real acceleration, demonstrating their effects on the final reading.

$$a = K \cdot A + b \quad \text{with} \quad K = \begin{bmatrix} k_x & 0 & 0 \\ 0 & k_y & 0 \\ 0 & 0 & k_z \end{bmatrix} \quad , \quad b = \begin{bmatrix} b_x \\ b_y \\ b_z \end{bmatrix} \quad (1)$$

Where A is the vector containing the true accelerations on the three axis and a represents the measured ones.

So, if the attitude angles of the sensor are known it is possible to compute the real values of accelerations that must be felt by the accelerometer A , resulting in a linear system. The linearity of this measurement model allows us to solve the problem for the estimation of bias (b_i) and scale factors (k_i) with a linear estimation algorithm since actually it is only a matter of solving a linear system.

$$a_i = K \cdot A_i + b \quad (2)$$

Although solving linear problems is computationally convenient, this model presents an important drawback: it is attitude-dependent. This means that to apply it for calibrations, extremely precise measures of the attitude angles during measurement process are required. A very small error in the inclination of the surface on which the accelerometer is placed or a human error on sensor alignment can lead to an incorrect estimation of the sensor's intrinsic errors. In such cases, it becomes impossible for the algorithm to distinguish between internal deterministic errors and external misalignment.

Since this vulnerability can lead to highly unreliable estimation and calibration processes, another measurement model is proposed:

$$A = s \cdot a + o \quad \longrightarrow \quad \begin{cases} A_x = s_x \cdot a_x + o_x \\ A_y = s_y \cdot a_y + o_y \\ A_z = s_z \cdot a_z + o_z \end{cases} \quad (3)$$

Since this model is the mathematical inverse of the previous one, the scale factors s_i and the bias o_i are denoted with different letters as their numerical values will differ from k_i and b_i .

The new model computes the true accelerations considering biases and scale factors applied to the measured one. More importantly, since the accelerometer only feels the constant gravitational acceleration $g = 9.81m/s^2$, it is possible to introduce a physical constraint on the norm of the total acceleration vector.

$$\|A\| = g \quad \longrightarrow \quad \sqrt{A_x^2 + A_y^2 + A_z^2} = g \quad (4)$$

This method allows to estimate the calibration parameters in an attitude-independent way, however, the constraint equation is non-linear, meaning that it must be solved with an appropriate non-linear optimization technique.

It is now clear that the trade-off for accurately and robustly estimating the scale factors and the biases is the necessity of solving a non-linear system. Consequently, this work will firstly present the linear estimation method design, followed by the non-linear one, concluding with a comparison between the results of the two approaches.

3 Linear Calibration Method

The design of the Linear Calibration Algorithm relies on the attitude-dependent measurement model introduced in 1. The core assumption of this method is that the exact orientation of the sensor relative to the local gravity vector is perfectly known for each measurement, which in a laboratory is typically achievable using precise multi-axes rotary tables.

3.1 True Acceleration Computation

To construct the linear system, the true acceleration A acting on the sensor must be computed for a set of N different known orientations. In a static condition, the only external specific force acting on the accelerometer is the local gravity vector, defined as $g_n = [0, 0, g]^T$. So to find the true acceleration projected on the sensor's body frame, the gravity vector must be multiplied by the rotation matrix R_n^b , which depends on the known attitude angles: roll(ϕ), pitch(θ), yaw(ψ).

$$A = R_n^b * \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} = \begin{bmatrix} -g \sin(\theta) \\ g \sin(\phi) \cos(\theta) \\ g \cos(\phi) \cos(\theta) \end{bmatrix} \quad (5)$$

Since the local gravity vector is always parallel to the Z axis in the navigation reference frame, the yaw axis (ψ) does not affect the static measurement.

So by placing the sensor in N different known position it is possible to compute exactly N true acceleration vectors $A_j = [A_{xj}, A_{yj}, A_{zj}]^T$ for $j=1, \dots, N$.

3.2 Accelerometer Measurement Simulation

To correctly validate the linear calibration algorithm, a realistic dataset must be generated. The simulation is designed to replicate the behavior of a commercial accelerometer datasheet (ADXL345)

k	1.021
b	0.196 m/s^2
σ	0.027 m/s^2

Table 1: Accelerometer data from ADXL345 Datasheet

where k is the scale factor, b is the zero-g offset or bias and σ is the standard deviation for the white gaussian noise model which can represent real noise on the measurement.

So the simulated measurement dataset will be obtained as:

$$a = K_{true} \cdot A + b_{true} + v_{noise} \quad (6)$$

where K_{true} is the diagonal matrix of scale factors, b_{true} is the bias column vector and $v_{noise} \sim \mathcal{N}(0, \sigma_{noise}^2)$ is the column vector which represents the stochastic error component. It is important to note that to ensure a realistic model, the single values k_i and b_i are different for each axis and slightly differ from those shown in Tab.1

To simulate the static calibration procedure realistically, the sensor is assumed to be held stationary at each position for a specific time window, creating stable data plateaus over which the noise can be averaged.

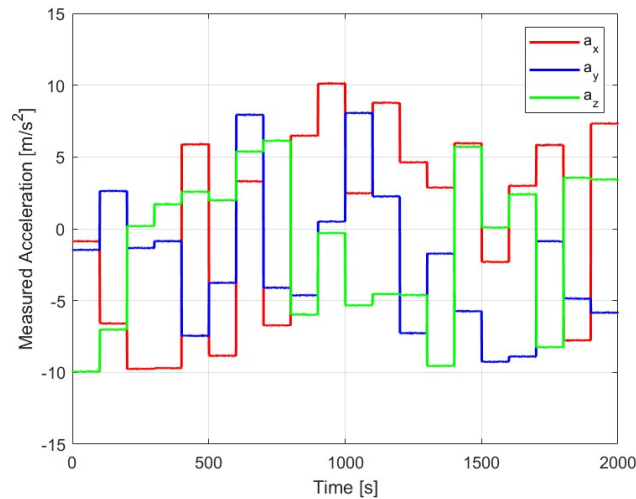


Figure 1: Accelerometer Measurement Simulation

3.3 Ordinary Least Square Algorithm

Algorithm Formulation The Ordinary Least Square Algorithm (OLS) is suitable for overdetermined systems in which it is possible to define a residual error term. Practically, the mathematical formulation of the system must be:

$$\mathbf{Y} = \mathbf{H}\mathbf{x} + \mathbf{v} \quad (7)$$

where $\mathbf{v}$ is the residual error term.

The goal of this algorithm is to find the state vector $\hat{\mathbf{x}}$ which minimizes the residual error. More precisely, it looks for the minimization of the sum of the square residuals.

So it is possible to express the cost function $\mathbf{J}$ as the scalar product between the residual and its transpose:

$$\mathbf{J}(\mathbf{x}) = \mathbf{v}^T \mathbf{v} = (\mathbf{Y} - \mathbf{H}\mathbf{x})^T (\mathbf{Y} - \mathbf{H}\mathbf{x}) \quad (8)$$

Developing the matrix product:

$$\mathbf{J}(\mathbf{x}) = \mathbf{Y}^T \mathbf{Y} - 2\mathbf{x}^T \mathbf{H}^T \mathbf{Y} + \mathbf{x}^T \mathbf{H}^T \mathbf{H} \mathbf{x} \quad (9)$$

To find the minimum of this cost function, the gradient with respect to the state vector $\mathbf{x}$ must be computed and set equal to zero:

$$\frac{\partial \mathbf{J}}{\partial \mathbf{x}} = -2\mathbf{H}^T \mathbf{Y} + 2(\mathbf{H}^T \mathbf{H})\mathbf{x} = \mathbf{0} \quad (10)$$

Dividing by 2 and isolating $\mathbf{x}$ it is possible to obtain the final equation of OLS algorithm.

$$\mathbf{x} = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T \mathbf{Y} \quad (11)$$

Attitude Profile Generation As can be seen from Eq.11 the performance of the Ordinary Least Square estimator strongly depends on matrix $\mathbf{H}$, in particular, the matrix $(\mathbf{H}^T \mathbf{H})$ must be invertible and so well-conditioned, to avoid amplification of the measurement noise. So if the selected attitude angles (ϕ and θ) are linearly dependent or explore only a small set of possible attitudes the $\mathbf{H}$ matrix can become ill-conditioned. To prevent this, a randomized attitude profile is explored by generating randomly the roll (ϕ) and pitch (θ) angles from a uniform distribution across their operational ranges.

$$\begin{cases} \phi \sim \mathcal{U}(-180^\circ, 180^\circ) \\ \theta \sim \mathcal{U}(-90^\circ, 90^\circ) \end{cases} \quad (12)$$

By generating N sufficiently spaced random static position, the attitude space is explored properly, ensuring a well conditioned system. This allows to consider the OLS as the best linear unbiased estimator, since avoids the necessity for regularization methods (e.g. Tikhonov Regularization) which are needed when matrix $\mathbf{H}$ is ill-conditioned and so matrix $(\mathbf{H}^T \mathbf{H})$ can be nearly singular.

OLS Application Since cross-axis misalignment and coupling are neglected in this fundamental model, the calibration can be decoupled into three independent linear optimization problems, one for each axis.

Taking the x-axis as an example, the measurement equation for the j -th position is:

$$a_{xj} = k_x A_{xj} + b_x \quad (13)$$

By stacking the equations for all N positions, the problem can be formulated in a matrix form $\mathbf{Y} = \mathbf{H}\mathbf{x}$ following what explained in 3.3.

$$\begin{bmatrix} a_{x1} \\ a_{x2} \\ \vdots \\ a_{xN} \end{bmatrix} = \begin{bmatrix} A_{x1} & 1 \\ A_{x2} & 1 \\ \vdots & \vdots \\ A_{xN} & 1 \end{bmatrix} \begin{bmatrix} k_x \\ b_x \end{bmatrix} \quad (14)$$

where $\mathbf{Y}$ is the $N \times 1$ vector containing the raw measurement from the sensor, $\mathbf{H}$ is the $N \times 2$ matrix containing the theoretical true accelerations computed via the rotation matrix and $\mathbf{x}$ is the 2×1 vector of the unknowns.

Because $N > 2$ the system is overdetermined. The optimal estimation of the scale factor and bias that minimizes the sum of square residuals can be found using the Ordinary Least Square (OLS) closed form solution presented in 3.3 Eq.11:

$$\mathbf{x} = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T \mathbf{Y} \quad (15)$$

The same procedure is repeated independently for y-axis and z-axis, replacing columns of $\mathbf{Y}$ and $\mathbf{H}$ with the respective measured and true accelerations for those axes, allowing the full population of the scale factor matrix $\mathbf{K}$ and bias vector $\mathbf{b}$.

3.4 Results

To evaluate the performance of the Ordinary Least Square (OLS) calibration algorithm, the estimated parameters are compared against the true values used during the dataset generation. Furthermore, the overall effectiveness of the calibration is verified by analyzing the Root Mean Square (RMSE) and the residuals plots.

Parameters Estimation The numerical results of the OLS estimation are summarized in Tab.2 and as can be seen, the estimation of the scale factors (k_i) is highly accurate, since any error in these parameters is heavily penalized by the optimization cost function. The estimated biases (b_i) also closely match the true values, with small deviations completely justified by the presence of the stochastic noise in the measurements.

Parameter	True Value	Estimated (OLS)	Absolute Error
X-Axis			
Scale Factor k_x	1.0200	1.0202	$2 \cdot 10^{-4}$
Bias $b_x [m/s^2]$	0.1300	0.1224	$7.6 \cdot 10^{-3}$
Y-Axis			
Scale Factor k_y	0.9900	0.9919	$1.9 \cdot 10^{-3}$
Bias $b_y [m/s^2]$	0.1600	0.1744	$14.4 \cdot 10^{-3}$
Z-Axis			
Scale Factor k_z	1.0500	1.0486	$1.4 \cdot 10^{-3}$
Bias $b_z [m/s^2]$	0.1500	0.1581	$8.1 \cdot 10^{-3}$

Table 2: Comparison between true and estimated parameters using the OLS algorithm.

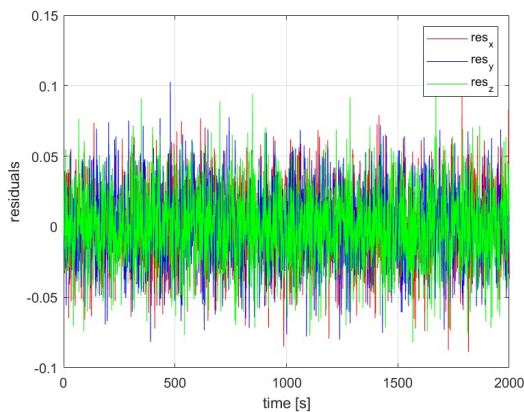
RMSE Analysis To provide a single global validation metric for the calibration process, the RMSE of the measured acceleration norm was calculated with respect to the theoretical gravity magnitude ($g = 9.81m/s^2$).

Before calibration, the raw simulated sensor exhibited an initial error of $RMSE_{pre} = 0.30m/s^2$, mainly driven by the zero-g offset or bias.

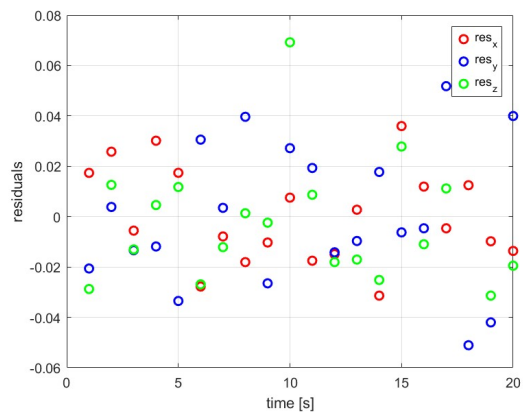
After applying the calibration with the estimated scale factors and biases, the error decreases dramatically to $RMSE_{post} = 0.026m/s^2$, which is aligned with the theoretical noise value of the sensor ($\sigma = 0.027m/s^2$).

Residual Analysis As previously cited in 3.3 the OLS algorithm allows to compute the residuals as: $\mathbf{v} = \mathbf{Y} - \mathbf{H}\mathbf{x}$. So a further validation for the linear model can be found in a graphical evaluation of the residuals in Fig.2.

In particular, Fig.2a shows the total OLS residuals computed across all the data samples. The plot exhibits a dense zero-mean noise band, confirming that the calibration parameters absorbed the systematic offse, leaving only the expected white gaussian noise.



(a) Residuals over complete data set



(b) Mean residuals over each plateau

Figure 2: Residual graphic evaluation

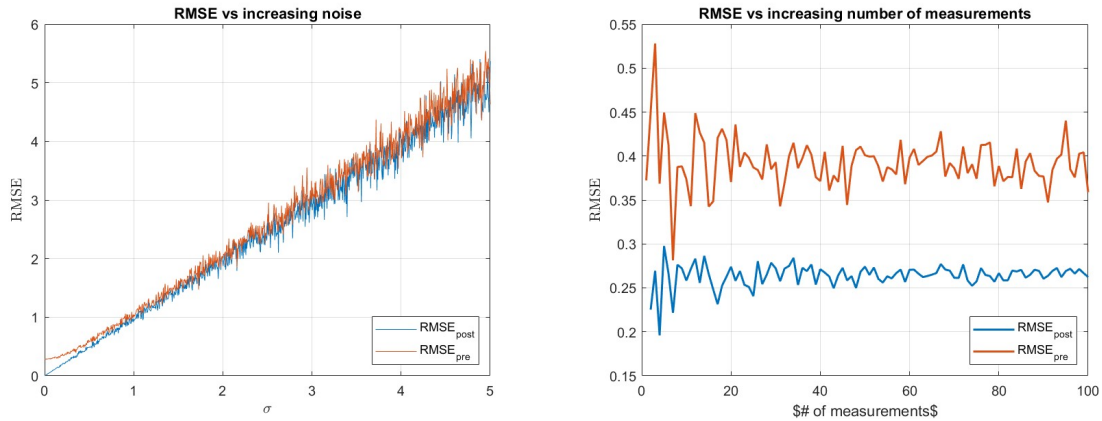
Similarly, Fig.2b displays the mean residuals computed over the static plateaus and as visible, the magnitude of the error is significantly reduced respect to the complete set of data. This happens because the noise is averaged over the entire plateau.

3.5 Sensitivity Analysis

To fully verify the robustness of the linear calibration algorithm, a sensitivity analysis was conducted using a Monte Carlo approach. The objective is to evaluate how different noise amplitudes and different number of attitude configurations impact the accuracy of the parameter estimation.

Impact of Noise Amplitude The first analysis looks at the degradation of the calibration accuracy with respect to the sensor's intrinsic noise level by keeping the number of static position fixed and progressively increasing the standard deviation of the white gaussian noise σ_{noise} .

As illustrated in Fig.3a, the post-calibration RMSE grows linearly with the increase in the noise amplitude, closely tracking the pre-calibration error. This behavior shows a fundamental theoretical limit also cited before in 3.3: while the OLS estimator successfully removes deterministic errors, it cannot eliminate stochastic noise. So the ultimate accuracy of the calibration and so of the estimated parameters is physically bounded by the noise level of the hardware.



(a) Impact of increasing noise on RMSE (b) Impact of # of attitude meas on stability

Figure 3: Montecarlo analysis for RMSE behavior with respect to increasing noise and different number of attitude measurements

Impact of Attitude Configurations The second test evaluates the impact of the number of attitude angles configurations on the calibration process by varying the number of randomly generated static measurements (N) used to define the matrix H and keeping constant the noise level.

As can be seen in Fig.3b this analysis reveals a critical mathematical vulnerability of the linear approach. For a low number of randomly generated static measurements

($N \sim < 10$), the RMSE exhibits severe instabilities and large spikes. This occurs because a limited or bad distributed set of attitude angles does not explore accurately the space, causing the columns of matrix H to become nearly linearly dependent, so the matrix $(H^T H)$ becomes badly scaled or close to singular, making its inversion highly sensitive to noise and leading to completely inaccurate parameter estimations.

However, as the number of randomly distributed measurements increases, the system becomes well-conditioned, the mathematical singularity disappears and the RMSE converges to a stable minimum corresponding to the sensor's true noise level.

4 Non-Linear Calibration Method

Unlike the linear approach, which requires precise knowledge of the sensor's attitude for every static measurement, the non-linear calibration relies entirely on a physical invariant: the magnitude of the measured static acceleration must perfectly match the local gravity g . This assumption completely removes the dependency on the rotation matrix, making the procedure robust and attitude-independent.

4.1 Cost Function Formulation

Recalling now the inverse measurement model, introduced in 2, where the true acceleration $\mathbf{A}$ can be expressed as a function of the raw measured acceleration $\mathbf{a}$:

$$\mathbf{A} = \mathbf{S}\mathbf{a} + \mathbf{o} \quad (16)$$

where $\mathbf{S} = \text{diag}(s_x, s_y, s_z)$ is the scale factor matrix and $\mathbf{o} = [o_x, o_y, o_z]^T$ is the offset vector. It is important to remember that the parameters s_i and o_i are defined in the inverse measurement model and they are not the same as the previous k_i and b_i . The state vector containing the six unknown calibration parameters to be estimated is defined as $\mathbf{x} = [s_x, s_y, s_z, o_x, o_y, o_z]^T$.

For any j -th static position, the ideal physical constraint dictates that $\|\mathbf{A}_j\| = g$, so it is possible to define a scalar residual error function $e_j(\mathbf{x})$, representing the deviation of the calibrated measurement's norm from the true gravitational acceleration:

$$e_j(\mathbf{x}) = \sqrt{(s_x a_{xj} + o_x)^2 + (s_y a_{yj} + o_y)^2 + (s_z a_{zj} + o_z)^2} - g \quad (17)$$

The goal of the non-linear calibration is to find the optimal parameter vector $\hat{\mathbf{x}}$ that minimizes the sum of the squared residuals over all N measurements.

The overall objective cost function $J(\mathbf{x})$ is thus formulated as:

$$J(\mathbf{x}) = \sum_{j=1}^N e_j^2(\mathbf{x}) \quad (18)$$

4.2 Optimization Method

Since the cost function $J(\mathbf{x})$ is highly non-linear with respect to the calibration parameters, finding a closed-form analytical solution like the one used in the OLS method is impossible and iterative optimization techniques must be employed.

Standard numerical approaches for solving non-linear least squares problems include for example the Gauss-Newton or the Levenberg-Marquardt algorithms. These iterative solvers require an initial guess for the state vector, typically set to ideal conditions ($\mathbf{S} = \mathbf{I}_{3 \times 3}$ and $\mathbf{o} = \mathbf{0}_{3 \times 1}$), and progressively update the parameters by descending along the gradient of the cost function until a stable local minimum is reached.

Algorithm Choice To minimize a non-linear function, both the Gauss-Newton and the Levenberg-Marquardt algorithms are good option but still there are difference between the two. More specifically, the Levenberg-Marquardt algorithms modifies the Gauss-Newton by adding a damping factor λ , which acts as a regularization term and so in practice, when λ is big, the algorithm passes from GN to Descent-Gradient. This approach is highly required when the initial guess is very far from the true solution or when the information matrix $\mathbf{J}^T \mathbf{J}$ is ill-conditioned and nearly singular.

However, neither of this problems applies to the considered experimental setup. In 3.4 has been explained that to have a reliable data set for calibration a sufficient high number of measurements is needed. As a consequence, also during this analysis the randomly generated different attitudes will be enough to have a well conditioned matrix. Furthermore, by initializing the state vector as $\mathbf{x}_0 = [1, 1, 1, 0, 0, 0]^T$ the starting point strictly resides within the local region of the global minimum.

Under these conditions, for the most of the cases in general, and for all the real cases (in which an high number of measurements is available) the dumping parameter λ introduced by the Levenberg-Marquardt algorithm is not necessary, so the Gauss-Newton algorithm is selected to minimize the non-linear cost function of interest.

Gauss-Newton Algorithm Derivation To iteratively minimize the non-linear cost function, the Gauss-Newton method relies on a first-order Taylor expansion of the residual vector. Let $E(\mathbf{x}) = [e_1(\mathbf{x}), e_2(\mathbf{x}), \dots, e_N(\mathbf{x})]^T$ be the error vector evaluated at the current state estimate $\mathbf{x}_K$. The residual vector can be linearly approximated as:

$$E(\mathbf{x}_k + \Delta \mathbf{x}) \approx E(\mathbf{x}_k) + \mathbf{J} \Delta \mathbf{x} \quad (19)$$

where $\mathbf{J}$ is the $N \times 6$ Jacobian Matrix containing the partial derivatives of the residuals with respect to the state variables, evaluated at $\mathbf{x}_k$.

The objective of each iteration of the algorithm is to find the step $\Delta \mathbf{x}$ which minimizes the norm of the linearized residual vector. So, setting the derivatives of

$\|E(\mathbf{x}) + \Delta\mathbf{x}\|^2$ with respect to $\Delta\mathbf{x}$ to zero is possible to obtain the normal equation for the linearized system:

$$\frac{\partial \|E(\mathbf{x}) + \Delta\mathbf{x}\|^2}{\partial \Delta\mathbf{x}} = 0 \quad \longrightarrow \quad (\mathbf{J}^T \mathbf{J}) \Delta\mathbf{x} = -\mathbf{J}^T E(\mathbf{x}) \quad (20)$$

Solving this system for $\Delta\mathbf{x}$ leads to the fundamental iterative update rule for the Gauss-Newton algorithm;

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta\mathbf{x} = \mathbf{x}_k - (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T E(\mathbf{x}) \quad (21)$$

Gauss-Newton Algorithm Application At this point, starting from the simulated accelerations data set presented in 3.2 and the measurement model cited in 4.1 the Gauss-Newton algorithm can be applied.

The starting point is the formulation of the Jacobian matrix $\mathbf{J}$, since performance and stability of the algorithm highly depend on it. Following the measurement model previously introduced, the current j -th acceleration measurement can be defined as:

$$\begin{cases} A_{xj} = s_x a_{xj} + o_x \\ A_{yj} = s_y a_{yj} + o_y \\ A_{zj} = s_z a_{zj} + o_z \end{cases} \quad (22)$$

and the estimated acceleration norm is:

$$\|\mathbf{A}_j\| = \sqrt{A_{xj}^2 + A_{yj}^2 + A_{zj}^2} \quad (23)$$

Since the residual is defined as $e_j(\mathbf{x}) = \|\mathbf{A}_j\| - g$ and the state vector, composed by the calibration parameters, is $\mathbf{x} = [s_x, s_y, s_z, o_x, o_y, o_z]^T$, the j -th row of the Jacobian matrix can be expressed as:

$$\mathbf{J}_j = \left[\frac{\partial e_j(\mathbf{x})}{\partial s_x}, \frac{\partial e_j(\mathbf{x})}{\partial s_y}, \frac{\partial e_j(\mathbf{x})}{\partial s_z}, \frac{\partial e_j(\mathbf{x})}{\partial o_x}, \frac{\partial e_j(\mathbf{x})}{\partial o_y}, \frac{\partial e_j(\mathbf{x})}{\partial o_z} \right] \quad (24)$$

where $e_j(\mathbf{x})$ is the residual expression presented in 4.1 Eq.17.

So, by applying the calculus chain rule, the partial derivatives can be computed. As an example, the partial derivatives for the X-axis scale factor s_x and offset o_x can be derived:

$$\frac{\partial e_j(\mathbf{x})}{\partial s_x} = \frac{A_{xj} \cdot a_{xj}}{\sqrt{A_{xj}^2 + A_{yj}^2 + A_{zj}^2}} \quad (25)$$

$$\frac{\partial e_j(\mathbf{x})}{\partial o_x} = \frac{A_{xj}}{\sqrt{A_{xj}^2 + A_{yj}^2 + A_{zj}^2}} \quad (26)$$

Then the partial derivatives for the Y and Z axes follow the exact same mathematical symmetry.

4.3 Results

The performance of the Gauss-Newton non-linear calibration algorithm was evaluated using the exact same metrics applied to the linear approach, ensuring a rigorous and fair comparison between the two methodologies.

Parameter Estimation and Model Translation As mentioned at the beginning of 4.1, it is fundamental to clarify that the non-linear objective function optimizes the state vector of the *inverse* measurement model, which are the inverse scale factors (**S**) and offsets (**o**), while the OLS method estimates the *direct* model parameters (**K** and **b**). So to allow a direct and mathematically sound comparison with the true dataset, the estimated non-linear parameters are translated into their equivalent direct counterparts using the following relationships:

$$k_i = 1/s_i \quad , \quad b_i = -o_i/s_i \quad (27)$$

As summarized in Tab.3, the equivalent parameters successfully match the true deterministic errors and the negligible discrepancies are strictly bounded by the stochastic noise variance, proving that the iterative algorithm successfully converged to the global minimum.

Parameter	True Value	Estimated (GN Equivalent)	Absolute Error
X-Axis			
Scale Factor k_x	1.0200	1.0199	$1 \cdot 10^{-4}$
Bias $b_x [m/s^2]$	0.1300	0.1294	$6 \cdot 10^{-4}$
Y-Axis			
Scale Factor k_y	0.9900	0.9901	$1 \cdot 10^{-4}$
Bias $b_y [m/s^2]$	0.1600	0.1584	$6 \cdot 10^{-4}$
Z-Axis			
Scale Factor k_z	1.0500	1.0503	$3 \cdot 10^{-4}$
Bias $b_z [m/s^2]$	0.1500	0.1498	$2 \cdot 10^{-4}$

Table 3: Comparison between true and equivalent direct parameters estimated via the Gauss-Newton algorithm.

By comparing the estimation results in Tab.3 with the ones in Tab.2 it is clear that the non-linear estimation returns much more precise estimated parameters with respect to the linear estimation method.

RMSE Analysis The global accuracy of the non-linear calibration was assessed by computing the Root Mean Square Error (RMSE) of the calibrated acceleration norm. The Gauss-Newton algorithm successfully reduced the initial raw error from 0.30 to a final post-calibration value of 0.026.

This final metric perfectly mirrors the theoretical noise floor of the sensor ($\sigma_{noise} = 0.027$) and is virtually identical to the optimal result achieved by the linear OLS method. This demonstrates a crucial engineering milestone: the non-linear algorithm

can fully compensate for systematic errors with state-of-the-art accuracy, without requiring any external attitude reference or expensive positioning equipment.

Residual Analysis A graphical validation of the convergence is provided by Fig.4 illustrating the final residuals E computed after the convergence of the algorithm, which represent the exact deviation of the calibrated acceleration norm from the theoretical gravity vector.

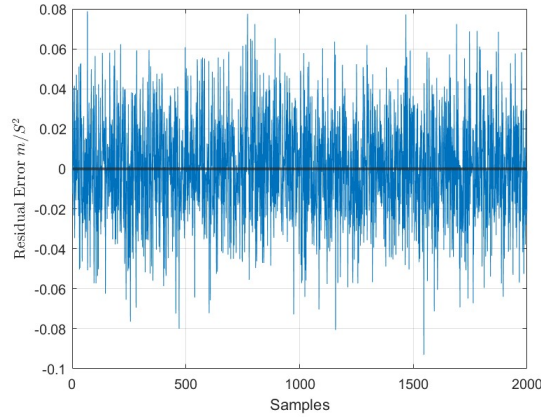


Figure 4: Final Gauss-Newton residuals

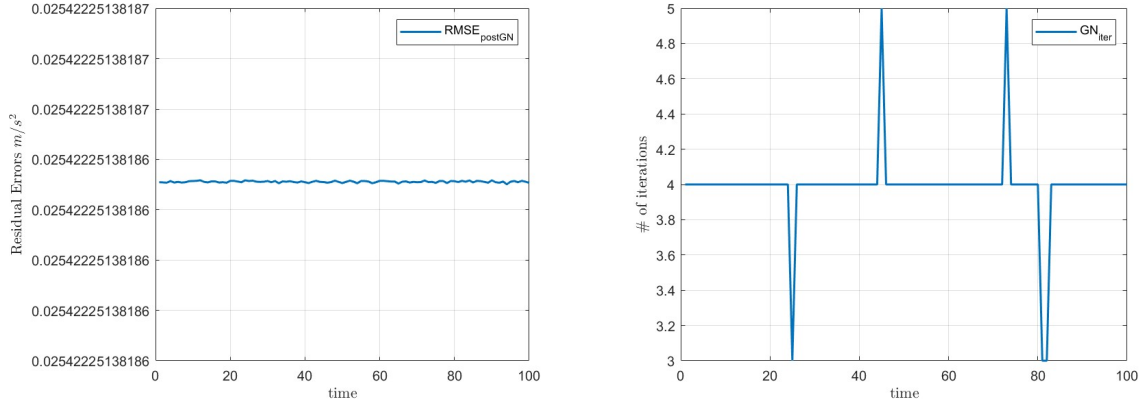
The plot exhibits a dense, zero-mean stochastic noise band with no visible structural or geometric trends. This confirms that all deterministic biases and scale factor errors have been entirely absorbed by the estimated parameters and so the remaining part is only about the additive white gaussian noise inherent to the sensor.

4.4 Sensitivity Analysis

To rigorously compare the Gauss-Newton approach against the linear OLS method, a similar Monte Carlo sensitivity analysis has been conducted to monitor the post-calibration RMSE and the number of iterations required to reach the convergence tolerance.

Impact of Different Initial Guess An analysis of the algorithm performance with respect to the choice of different initial guess has been conducted in order to verify robustness of the implemented algorithm.

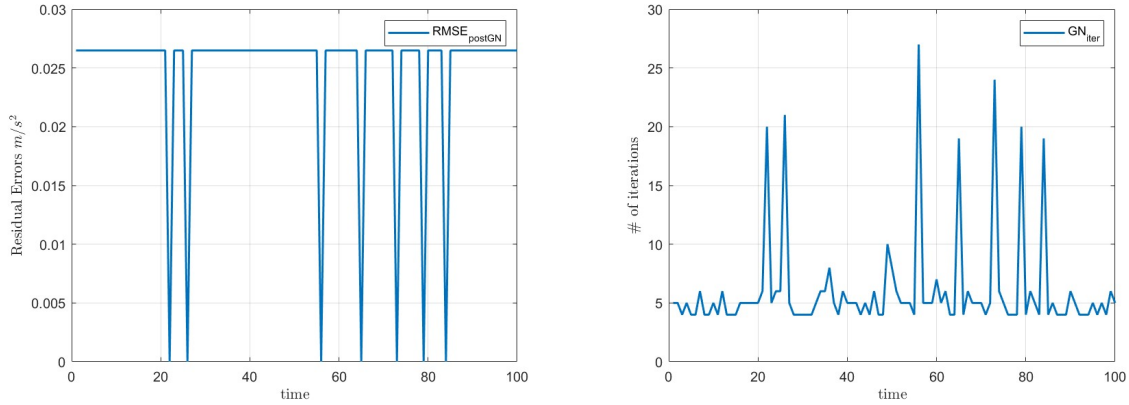
For the first test has been considered a series of randomly generated initial guess in the range of 30% for both scale factors and biases and as can be seen in Fig.5 the algorithm still works well, ensuring the best RMSE level post calibration for all the cases and reaching the convergence in a very small number of iterations (3 to 5).



(a) Impact of different initial guess on RMSE (b) Impact of different initial guess on GN iter

Figure 5: Montecarlo analysis for GN algo behavior with respect to different initial guess within a range of 30% of error in terms of RMSE and number of iterations

Then to further stress the robustness of the algorithm the possible error on the initial guess has been moved to 100%. In this case, it is possible to observe a mathematical vulnerability, since when the randomly generated initial guess for the scale factors approaches the value of zero, causing a reduction of Jacobian matrix rank and leading to a mathematical singularity. However, is important to remember that this case is a mathematical experiment to study the performance of the algorithm, however it is very far from real cases applications.



(a) Impact of different initial guess on RMSE (b) Impact of different initial guess on GN iter

Figure 6: Montecarlo analysis for GN algo behavior with respect to different initial guess within a range of 100% of error in terms of RMSE and number of iterations

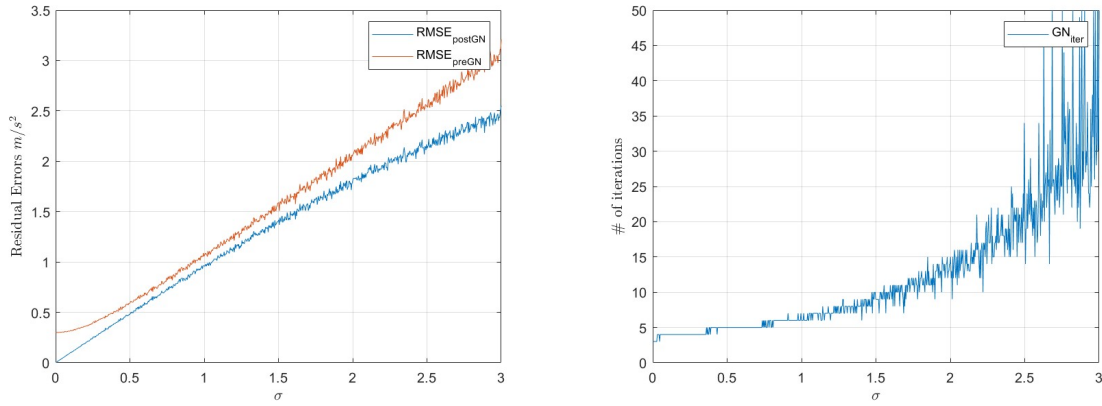
Impact of Increasing Noise Level The sensitivity analysis continues evaluating the RMSE and on the number of iterations of the algorithm with respect to an increase in the noise amplitude noise amplitude, shown in Fig.7.

Similarly to the linear OLS results, what is shown in Fig.7a is that the post-calibration RMSE grows linearly, bounded entirely by the inner stochastic noise of

the sensor. However, there is an important difference to note comparing this result to the linear case. Looking at Fig.3a after a first region where the calibration makes a lot difference with respect to the raw data in terms of RMSE, when the noise increase the two curves overlap, meaning that the OLS correctly leaves the uncompensable zero-mean noise intact.

In contrast, in Fig.7a is shown that post calibration RMSE remains much lower than the one of raw data not only in the first region, but for all noise values. This reveals a known vulnerability of the non-linear approach: the *noise rectification effect*. At high noise variances, the squaring operation inside the vector norm systematically increases the mean magnitude of the data and to compensate this fact, the GN algorithm shrinks the estimated scale factors to force the norm back to g_0 , effectively overfitting the noise and introducing estimation bias.

Following this, it is possible to state that while GN is highly effective under nominal conditions without requiring attitude knowledge, the OLS estimator structurally maintains a higher theoretical robustness against extreme stochastic noise.



(a) Impact of increasing noise on RMSE

(b) Impact of increasing noise on GN iter

Figure 7: Montecarlo analysis for GN algo behavior with respect to increasing noise in terms of RMSE and number of iterations

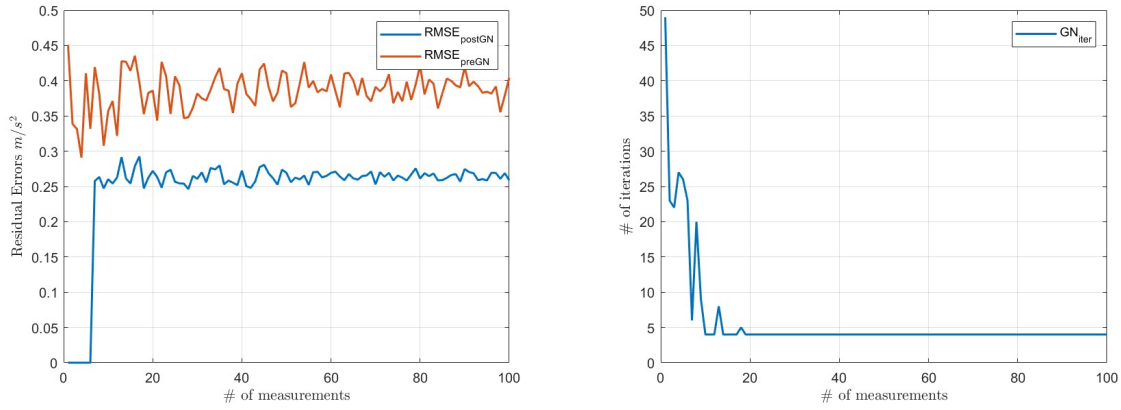
However, the analysis of the algorithm iterations number in Fig.7b reveals a critical phenomenon: as the noise amplitude increases, the objective cost function space loses its smooth convex shape, becoming progressively more scattered. This happens because the Gauss-Newton algorithm requires an exponentially higher and highly variable number of iterations to satisfy the strict convergence tolerance and so struggles to find the exact local minimum within the noisy data bounds.

Impact of Different Number of Attitude Measurements The last test for the sensitivity analysis is displayed in Fig.8 and consists in evaluating the robustness of the algorithm against the number of spatial attitudes (N) used to populate the Jacobian matrix $\mathbf{J}$.

The results perfectly highlight the mathematical vulnerability of the unregularized Gauss-Newton method explained in 4.2. For a small number of measurements ($N \sim <$

10), the geometric space is insufficiently explored, causing the information matrix ($\mathbf{J}^T \mathbf{J}$) to become ill-conditioned. As a consequence, in this region the post-calibration RMSE exhibits massive spikes, and the algorithm fails to converge efficiently, frequently hitting the maximum iteration limit (50 iterations).

On the other hand, once a sufficient number of spatially diverse measurements is provided ($N > \sim 15$), the Jacobian achieves full rank and excellent conditioning. So the mathematical singularity vanishes, the RMSE drops to the ideal noise level, and the algorithm demonstrates its theoretical rapid convergence, consistently locating the global minimum in just 4 to 5 iterations. Also in this case it is observed a similar behavior of the OLS algorithm.



(a) Impact of # of attitude meas on stability (b) Impact of # of attitude meas on GN iter

Figure 8: Montecarlo analysis for GN algo behavior with respect to different number of attitude measurements in terms of stability and number of iterations

5 Conclusions

This work presented a comprehensive study on the static calibration of a triaxial accelerometer, comparing a linear attitude-dependent approach (OLS) with a non-linear attitude-independent method (Gauss-Newton).

Both methods successfully achieved the sensor's theoretical accuracy limit, reducing the initial systematic error to a post-calibration RMSE of approximately $0.026 m/s^2$, which aligns with the physical hardware noise level.

The Ordinary Least Squares algorithm proved to be a computationally efficient and robust estimator, however, its reliance on precise attitude knowledge represents a significant operational constraint in real-world scenarios.

The Gauss-Newton algorithm successfully overcome this limitation by exploiting the physical invariance of the gravitational acceleration norm. The analysis demonstrated that the non-linear approach provides a more precise identification of the intrinsic sensor parameters (scale factors and biases) by decoupling them from external orientation errors. Furthermore, the Monte Carlo sensitivity analysis confirmed the robustness of

the algorithm, showing consistent convergence even when initialized with parameter errors up to $\pm 30\%$.

The study also identified the mathematical boundaries of these estimators: while OLS suffers from numerical instability when the attitude space is poorly explored, the Gauss-Newton method is susceptible to "noise rectification" and potential singularities if initialized with zero scale factors.

In conclusion, for modern industrial and aerospace applications where precise positioning equipment is often unavailable, the non-linear attitude-independent calibration stands out as the superior engineering choice, offering state-of-the-art accuracy combined with significant operational flexibility.